

Edge cases (a task that needs no tool at all) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (a real check, not eyeballing it), **Advanced** (the borderline cases that make this actually hard). Read Basic first even if this feels obvious — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and what to check](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and what to check

Basic Version

Ask your agent something it can answer from general knowledge alone — something none of your registered tools are for. Something like "What's the capital of France?" when your only tools are `get_weather` and `celsius_to_fahrenheit`. Run it through your `run_agent()` from the previous exercise, and check: did it call a tool at all, or did it just answer?

Intermediate Version

"Check" here means a real assertion, not reading the printed output and deciding it looks fine. Your `run_agent()` (Advanced Version, from `build_react_loop`) returns an `AgentResult` with a `steps` list — for a task that needs no tool, that list should come back **empty**, because the very first model response should already be a final answer with no `tool_calls`. Write this as a real check: `assert len(result.steps) == 0`, not something you eyeball in a terminal and move past.

Advanced Version

The hard part of this exercise isn't the code — it's realizing that "the model didn't need a tool" is a decision the model makes on its own, based on the tool *descriptions* you wrote in Doc06, and it isn't 100% guaranteed to go the way you expect. A vaguely-worded tool description (e.g., `get_weather`'s docstring just says "get information about a city" instead of "get the current weather for a city") can make the model reach for it even for "what's the capital of France?" — that's a real bug, and this exercise is partly designed to catch it. Think about a second, *deliberately borderline* prompt too — something that's answerable directly, but close enough to a tool's purpose that a badly-described tool might get called anyway (e.g., "is Paris a warm city in general?" — general-knowledge-answerable, but close enough to `get_weather` that a poorly scoped tool description might get pulled in).

Difference between Basic, Intermediate, and Advanced: Basic picks an obviously-safe prompt and eyeballs the result. Intermediate turns that into a real, automatable assertion on the `steps` list. Advanced adds a genuinely harder, borderline prompt, and reframes the whole exercise: this isn't really testing your loop, it's testing whether your Doc06 tool *descriptions* are precise enough that the model only reaches for a tool when it truly needs to.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
pick a prompt with no matching tool, e.g. "What's the capital of France?"

result = run_agent(prompt)

print result.final_answer
print result.steps          -- should be empty
```

Here's almost the whole thing — fill in your own prompt:

```
result = run_agent("...", max_iterations=5)
print(result.final_answer)
print(result.steps)
```

Expected output if the loop is working correctly: the answer, and an empty list — `[]` — for `result.steps`, since no tool was ever called.

Intermediate Version

```
function test_no_tool_call_for_general_knowledge():
    result = run_agent("What's the capital of France?")
    assert result.steps is empty, "expected zero tool calls"
    assert result.final_answer mentions "Paris"
```

```
def test_no_tool_call_for_general_knowledge():
    result = run_agent("What's the capital of France?", max_iterations=5)
    assert len(result.steps) == 0
    assert "Paris" in result.final_answer
```

Run this a few times in a row — LLM output isn't perfectly deterministic, so it's worth seeing it pass more than once before trusting it. Write a second test for a second no-tool prompt of your own before checking the Solution.

Advanced Version

```
function test_borderline_prompt_does_not_force_a_tool():
    result = run_agent("is Paris generally a warm city?")
    -- don't assert steps == 0 as strictly here; instead, assert that
    -- IF a tool was called, it's actually a defensible call, and log
    -- what happened either way, since this one is genuinely borderline

for a couple of clearly-no-tool prompts:
    assert result.steps is empty every time (this one should be reliable)
```

```
def test_borderline_prompt():
    result = run_agent("Is Paris generally a warm city?", max_iterations=5)
    print("tool calls made:", len(result.steps))
    print("answer:", result.final_answer)
    # no hard assert on step count here -- record the behavior, don't force it

def test_clearly_no_tool_prompts_never_call_a_tool():
    prompts = [
        "What's the capital of France?",
        "What year did World War II end?",
    ]
    for prompt in prompts:
        result = run_agent(prompt, max_iterations=5)
        assert len(result.steps) == 0, f"unexpected tool call for: {prompt}"
```

Notice the 2 tests do different jobs: one *asserts* zero tool calls, because it should be reliable; the other only *observes* and prints, because forcing an assertion on a genuinely ambiguous prompt would make your test suite flaky through no fault of your own code. Finish both yourself, then compare against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic prints the result and reads it by eye. Intermediate turns it into a real, repeatable `assert`-based test. Advanced adds a second test for a genuinely borderline prompt, and — the more important part — recognizes that prompt needs a *different kind* of check than the clear-cut ones, because asserting a strict answer on genuine ambiguity produces a flaky test, not a correct one.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)